# Sprawozdanie – część 3

Sklep internetowy **KawaNaKlik**
Część A: APPDB – Etap II: Automatyzacja przepływów (bez AI)
Część B: CMS – Laboratorium 2: ACF, własne typy treści i bloki

| | |
|---|---|
| **Zespół (2 os.)** | 1. .....<br>2. ..... |
| **Scenariusz** | Sklep internetowy – zamówienia, stany magazynowe, treści strony |
| **Repozytorium** | github.com/Mateuszl28/KawaNaKlik/tree/main/LAB03 (tag `lab2`) |
| **Środowisko** | Local · WordPress 7.1.3 · WooCommerce 11.2.1 · Secure Custom Fields · PHP 8.2.29 · n8n 2.45.0 · Mailpit |
| **Data** | 23.10.2026 |

# Spis treści

**Część A – APPDB: Automatyzacja przepływów (bez AI)**

A1. Realizacja zadań

A2. Wprowadzenie do n8n

A3. Architektura rozwiązania

A4. WF1 – przepływ zamówienia

A5. WF2 – kontrola stanów magazynowych (cron)

A6. Obsługa błędów i logowanie

A7. Diagramy procesu docelowego (TO-BE)

A8. Testy

A9. Napotkane problemy

A10. Efekt i wnioski

**Część B – CMS: ACF, własne typy treści i bloki**

B1. Realizacja zadań

B2. ACF Local JSON – pola w repozytorium

B3. Własne typy treści

B4. Bloki ACF

B5. Ograniczenie listy bloków i dane globalne

B6. Odpowiedzi na pytania

B7. Problemy i rozwiązania

B8. Wnioski

**Załączniki – linki do repozytorium**

# Część A – APPDB

Etap II: Automatyzacja przepływów (bez AI)

## A1. Realizacja zadań

| Zadanie | Realizacja | Status |
|---|---|---|
| Wprowadzenie do n8n: węzły, wyzwalacze, wyrażenia, obsługa błędów | Samodzielnie hostowane n8n 2.45 na komputerze zespołu. Opis pojęć z przykładami z naszych przepływów: rozdz. A2. | ✓ |
| Pierwszy przepływ: webhook → zapis w arkuszu → walidacja → e-mail → Slack/Discord | **WF1** (22 węzły): webhook WooCommerce `order.created` z weryfikacją podpisu HMAC, walidacja danych, zapis wiersza w arkuszu zamówień, e-mail z potwierdzeniem do klienta, powiadomienie na kanale Discord. Ścieżki błędne: dane niekompletne (422) i zły podpis (401). | ✓ |
| Drugi przepływ zaplanowany w czasie (cron) | **WF2** (16 węzłów): codziennie o 07:00 pobiera produkty z API WooCommerce i wysyła raport o stanach ≤ 5 szt. e-mailem do magazynu i alertem na Discord. Można go też uruchomić na żądanie (GET). | ✓ |
| Obsługa błędów i logowanie (API nie odpowiada, dane niekompletne) | Ponowienia (3 próby co 3 s) i wyjście błędu w węźle API, osobne ścieżki walidacji i podpisu, przepływ **WF3** z wyzwalaczem Error Trigger, limit czasu wykonania 120 s, log błędów w pliku CSV i historia wykonań n8n. | ✓ |
| Diagram procesu docelowego TO-BE | Dwa diagramy BPMN (draw.io + PNG): realizacja zamówienia i kontrola stanów (rozdz. A7). | ✓ |
| Do sprawozdania: zrzuty, eksport JSON, TO-BE, testy (w tym błędne), problemy | 17 zrzutów ekranu, eksport JSON 4 przepływów, 9 przypadków testowych (6 błędnych) uruchamianych jednym skryptem, opis 8 problemów. | ✓ |

> **Zamienniki usług zewnętrznych.** Pracujemy wyłącznie lokalnie, na danych syntetycznych i bez kont w usługach chmurowych. Dlatego:
>
> - **Arkusz zamówień** to plik CSV dopisywany przez n8n (otwiera się w Excelu). Zastąpienie go węzłem Google Sheets to zmiana jednego węzła i dodanie poświadczenia OAuth.
> - **Discord** zastępuje przepływ WF0, który przyjmuje dokładnie ten sam JSON co webhook Discorda ( `username` , `content` ) i zapisuje go do pliku. Prawdziwy kanał włącza się jedną zmienną środowiskową `DISCORD_WEBHOOK_URL` .
> - **Poczta** idzie przez lokalny serwer SMTP **Mailpit**, który przechwytuje wszystkie wiadomości.

## A2. Wprowadzenie do n8n

**n8n** to narzędzie do budowania automatyzacji z bloków (low-code). Przepływ (*workflow*) to graf węzłów połączonych strzałkami. Każdy węzeł dostaje listę elementów (*items*) w formacie JSON, wykonuje jedną operację i przekazuje wynik dalej. n8n można uruchomić u siebie (open source, model *fair-code*), dzięki czemu dane klientów nie opuszczają serwera firmy.

| Pojęcie | Co to jest | Przykład w naszych przepływach |
|---|---|---|
| **Węzeł** (node) | Pojedynczy krok: integracja z usługą, operacja na danych, warunek, kod. Węzeł ma wejścia i wyjścia; węzły warunkowe (IF) mają po dwa wyjścia: *true* i *false*. | Webhook, Code (JavaScript), IF, Edit Fields, Write File to Disk, Send Email, HTTP Request, Respond to Webhook. |
| **Wyzwalacz** (trigger) | Węzeł startowy, który uruchamia przepływ. Przepływ musi być *opublikowany* (aktywny), żeby wyzwalacz nasłuchiwał. | Webhook (zdarzenie z WooCommerce), Schedule Trigger z wyrażeniem cron `0 7 * * *`, Error Trigger (błąd w innym przepływie). |
| **Wyrażenie** (expression) | Fragment JavaScriptu w `{{ }}` wewnątrz pola węzła. Ma dostęp do danych bieżącego elementu (`$json`), danych innych węzłów (`$('Nazwa')`), daty (`$now`) i zmiennych środowiskowych (`$env`). | `{{ $('Weryfikacja i normalizacja').first().json.order.email }}`, `{{ $now.toFormat('yyyy-LL-dd HH:mm:ss') }}`, `{{ $env.DISCORD_WEBHOOK_URL \|\| '…mock' }}` |
| **Poświadczenia** (credentials) | Dane logowania do usług, przechowywane w n8n w postaci zaszyfrowanej, poza definicją przepływu. Eksport JSON przepływu zawiera tylko identyfikator poświadczenia. | SMTP (Mailpit), WooCommerce API (klucz i sekret REST). |
| **Obsługa błędów** | Na poziomie węzła: *Retry On Fail* (ponowienia), *On Error* (zatrzymaj, kontynuuj, kieruj do wyjścia błędu). Na poziomie przepływu: *Error Workflow* (osobny przepływ z Error Trigger), limit czasu wykonania, historia wykonań. | API WooCommerce: 3 próby co 3 s, potem wyjście „Error". Discord: 2 próby, przy błędzie przepływ idzie dalej. WF1 i WF2 mają ustawiony WF3 jako Error Workflow. |

## Środowisko

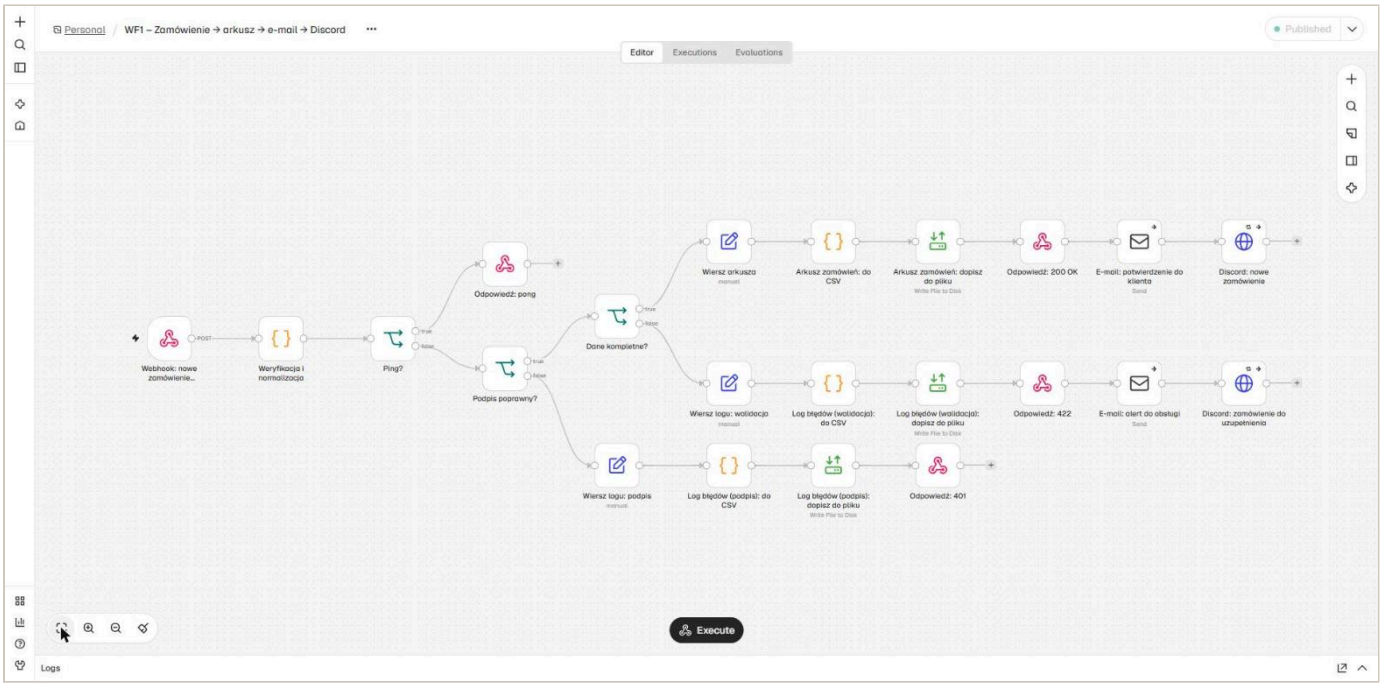
| Element | Konfiguracja |
|---|---|
| n8n 2.45.0 (Node.js 24) | Uruchamiany skryptem `kod/start_n8n.sh` pod adresem `http://localhost:5690`. Dane n8n (baza SQLite, zaszyfrowane poświadczenia) leżą w katalogu użytkownika, poza repozytorium. |
| Zmienne środowiskowe | `WC_WEBHOOK_SECRET` (sekret podpisu, czytany z pliku poza repozytorium), `NODE_FUNCTION_ALLOW_BUILTIN=crypto` (HMAC w węźle Code), `N8N_RESTRICT_FILE_ACCESS_TO` (n8n może zapisywać pliki tylko w katalogu z arkuszem i logami), `GENERIC_TIMEZONE=Europe/Warsaw`. |
| Sklep | WordPress 7.1.3 + WooCommerce 11.2.1 w aplikacji Local (`kawanaklik.local`). Webhook „Zamówienie utworzone" → `http://localhost:5690/webhook/zamowienie` z sekretem. Klucze REST API tylko do odczytu produktów. |
| Wtyczka MU `kawanaklik-dev-webhooks.php` | WooCommerce domyślnie nie wysyła webhooków na adresy lokalne i porty inne niż 80/443/8080. Wtyczka dopuszcza wyłącznie `localhost:5690` i wysyła webhooki od razu (synchronicznie), żeby testy nie czekały na WP-Cron. Tylko na środowisku deweloperskim. |

## A3. Architektura rozwiązania

| Przepływ | Wyzwalacz | Zadanie |
|---|---|---|
| **WF1** – Zamówienie → arkusz → e-mail → Discord | Webhook POST `/webhook/zamowienie` | Weryfikuje podpis, waliduje dane, zapisuje zamówienie w arkuszu, potwierdza klientowi, powiadamia zespół. Odpowiada sklepowi kodem 200 / 422 / 401. |
| **WF2** – Kontrola stanów magazynowych | Cron `0 7 * * *` + GET `/webhook/kontrola-stanow` | Pobiera produkty przez REST API WooCommerce, filtruje stany ≤ 5 szt., wysyła raport do magazynu i alert na Discord, zapisuje log kontroli. |
| **WF3** – Obsługa błędów | Error Trigger | Uruchamia się przy każdym nieobsłużonym błędzie WF1/WF2: wiersz w logu błędów, alert na Discord, e-mail do administratora. |
| **WF0** – Mock Discord (test) | Webhook POST `/webhook/discord-mock` | Udaje webhook Discorda: przyjmuje wiadomość i dopisuje ją do `discord_mock.csv`. |

Pliki wynikowe (w katalogu dozwolonym dla n8n): `arkusz_zamowien.csv`, `log_bledow.csv`, `log_kontroli_stanow.csv`, `discord_mock.csv`. Przepływy nie są „wyklikane" ręcznie: generuje je skrypt `kod/n8n_gen.py`, a do n8n trafiają poleceniem `n8n import:workflow`. Dzięki temu definicja przepływów jest w repozytorium, a zmiany widać w historii gita.

## A4. WF1 – przepływ zamówienia




Rys. A1. WF1 w edytorze n8n: ścieżka poprawna (góra), dane niekompletne (środek), zły podpis (dół).

1. **Webhook: nowe zamówienie** przyjmuje POST z WooCommerce. Opcja *Raw Body* zachowuje surową treść żądania, potrzebną do sprawdzenia podpisu.
2. **Weryfikacja i normalizacja** (węzeł Code):
   - rozpoznaje „ping", który WooCommerce wysyła przy zapisie webhooka,

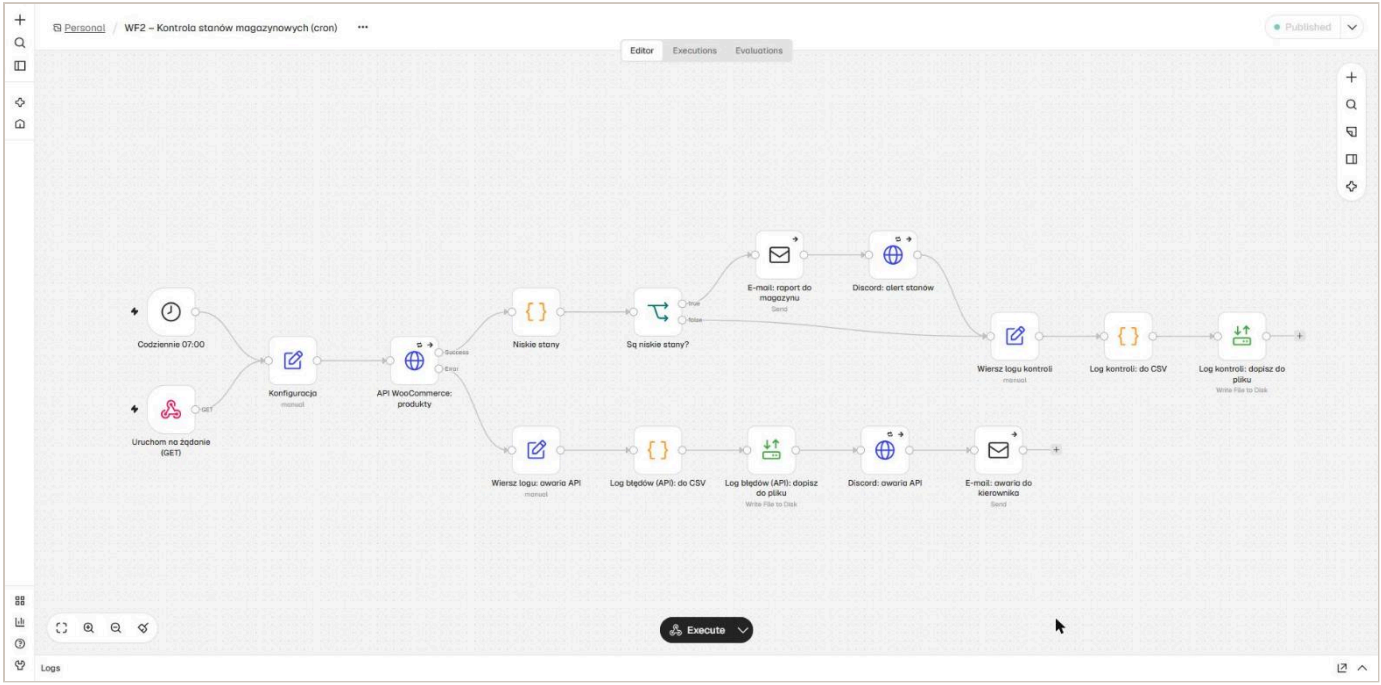
- liczy HMAC-SHA256 z surowej treści i porównuje go z nagłówkiem `X-WC-Webhook-Signature` funkcją `timingSafeEqual`, odporną na ataki czasowe,
- spłaszcza zamówienie do prostych pól: numer, klient, e-mail, adres, pozycje, kwota,
- buduje listę problemów walidacji.

3. **Ping?** → odpowiedź „pong". **Podpis poprawny?** → przy złym podpisie wiersz w logu błędów i odpowiedź **401**.
4. **Dane kompletne?** Sprawdzane są:
   - obecność i format adresu e-mail,
   - imię i nazwisko,
   - adres (ulica, kod, miasto),
   - co najmniej jedna pozycja,
   - kwota > 0.
5. **Ścieżka poprawna:** wiersz arkusza → dopisanie do `arkusz_zamowien.csv` → odpowiedź **200** do WooCommerce → e-mail z potwierdzeniem do klienta → wiadomość na Discord.
6. **Ścieżka „dane niekompletne":** wiersz w logu błędów → odpowiedź **422** z listą problemów → e-mail do działu obsługi klienta → alert na Discord. Zamówienie nie trafia do arkusza, dopóki ktoś nie uzupełni danych.

```
// fragment węzła „Weryfikacja i normalizacja"
const expected = crypto.createHmac('sha256', SECRET).update(raw).digest('base64');
const valid = SECRET !== '' && signature.length === expected.length &&
  crypto.timingSafeEqual(Buffer.from(signature), Buffer.from(expected));
...
if (!order.email) problemy.push('brak adresu e-mail');
else if (!/^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(order.email)) problemy.push('niepoprawny adres e-mail');
if (!b.address_1 || !b.city || !b.postcode) problemy.push('niepełny adres');
```

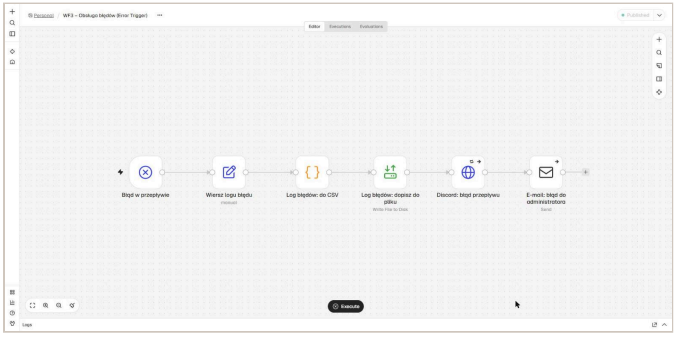
Odpowiedź do WooCommerce wysyłamy zaraz po zapisie w arkuszu, a dopiero potem e-mail i Discord. WooCommerce przy tworzeniu zamówienia czeka na odpowiedź webhooka (do 60 s), więc wolny serwer pocztowy nie może spowalniać sklepu (problem nr 5 w rozdz. A9).

## A5. WF2 – kontrola stanów magazynowych (cron)

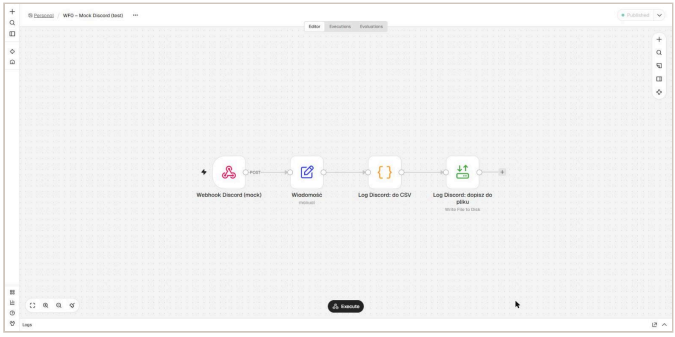



Rys. A2. WF2: dwa wyzwalacze (cron 07:00 i GET na żądanie), węzeł API z wyjściem „Success" i „Error".

- **Konfiguracja** (Edit Fields) trzyma parametry w jednym miejscu: próg (5 szt.), adres API, adres e-mail magazynu.
- **API WooCommerce: produkty** (HTTP Request z poświadczeniem WooCommerce) – `GET /wp-json/wc/v3/products?per_page=100&status=publish`, limit 10 s, *Retry On Fail*: 3 próby co 3 s, *On Error*: wyjście błędu.
- **Niskie stany** (Code) filtruje produkty ze śledzonym stanem ≤ progu, sortuje rosnąco i buduje tabelę HTML do e-maila oraz tekst na Discord.
- **Są niskie stany?** → raport e-mail do magazynu + alert Discord. W obu przypadkach powstaje wiersz w `log_kontroli_stanow.csv` (ile produktów sprawdzono, ile ma niski stan).
- **Wyjście „Error"** (API nie odpowiedziało po 3 próbach) → wiersz w logu błędów → alert Discord → e-mail do kierownika operacyjnego.



Rys. A3. WF3 – obsługa błędów (Error Trigger).

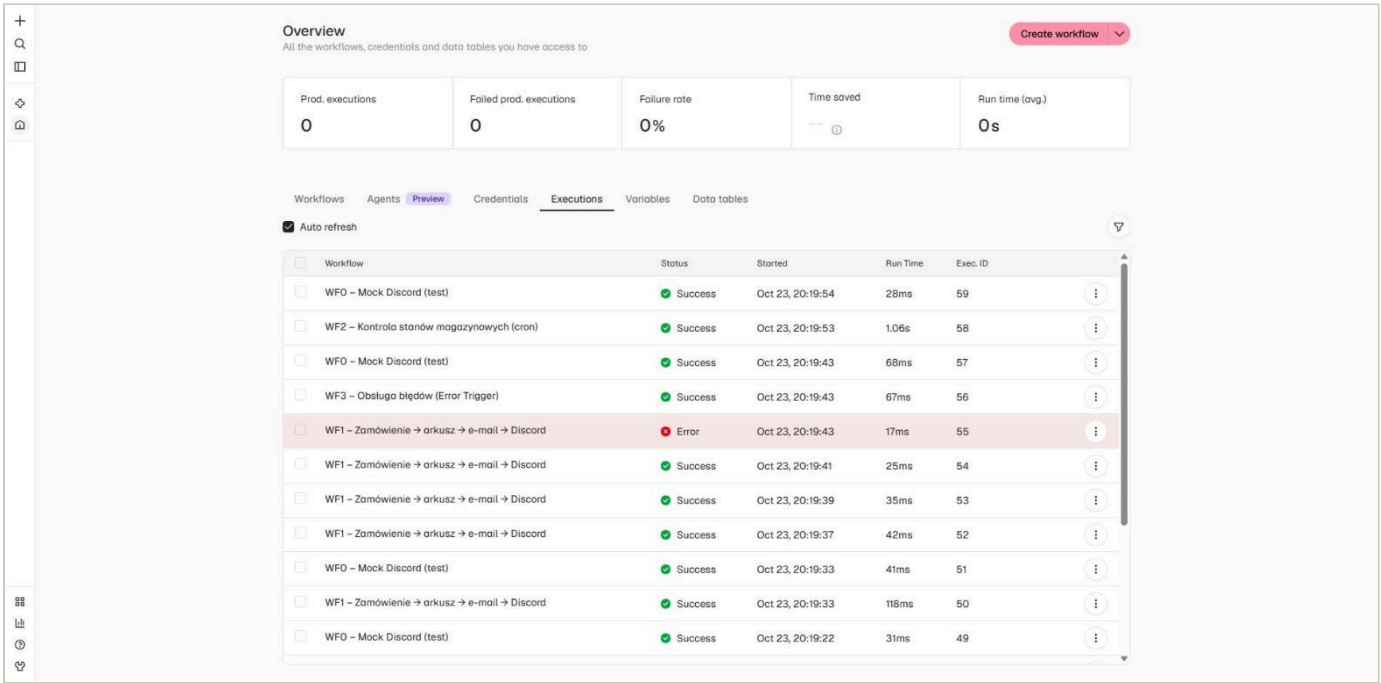


Rys. A4. WF0 – zastępczy webhook Discorda.

## A6. Obsługa błędów i logowanie

| Sytuacja | Co robi automat | Gdzie to widać |
|---|---|---|
| API WooCommerce nie odpowiada (serwer wyłączony, błąd 5xx, przekroczony czas) | 3 próby co 3 s, potem wyjście błędu: log, alert Discord, e-mail do kierownika. Kontrola powtórzy się następnego dnia o 07:00; można ją też uruchomić ręcznie. | `log_bledow.csv`, Discord, e-mail, historia wykonań |
| Dane zamówienia niekompletne lub błędne | Zamówienie nie trafia do arkusza. Log z listą problemów, alert do obsługi klienta (e-mail + Discord), odpowiedź 422 do sklepu. | `log_bledow.csv`, e-mail do obsługi |
| Fałszywe lub niepodpisane żądanie na adres webhooka | Odrzucenie (401) bez przetwarzania, wpis w logu z przyczyną. | `log_bledow.csv` |
| Nieprzewidziany wyjątek w dowolnym węźle | Wykonanie kończy się błędem, n8n uruchamia WF3: log z nazwą przepływu, węzła i numerem wykonania, alert Discord, e-mail do administratora. | WF3, historia wykonań |
| Discord niedostępny | 2 próby, potem przepływ idzie dalej (*Continue*). Brak powiadomienia nie blokuje zapisu zamówienia ani e-maila do klienta. | historia wykonań |
| Węzeł się zawiesza (np. połączenie SMTP) | Limit czasu wykonania przepływu: 120 s. Po jego przekroczeniu wykonanie kończy się błędem i trafia do WF3. | WF3, historia wykonań |

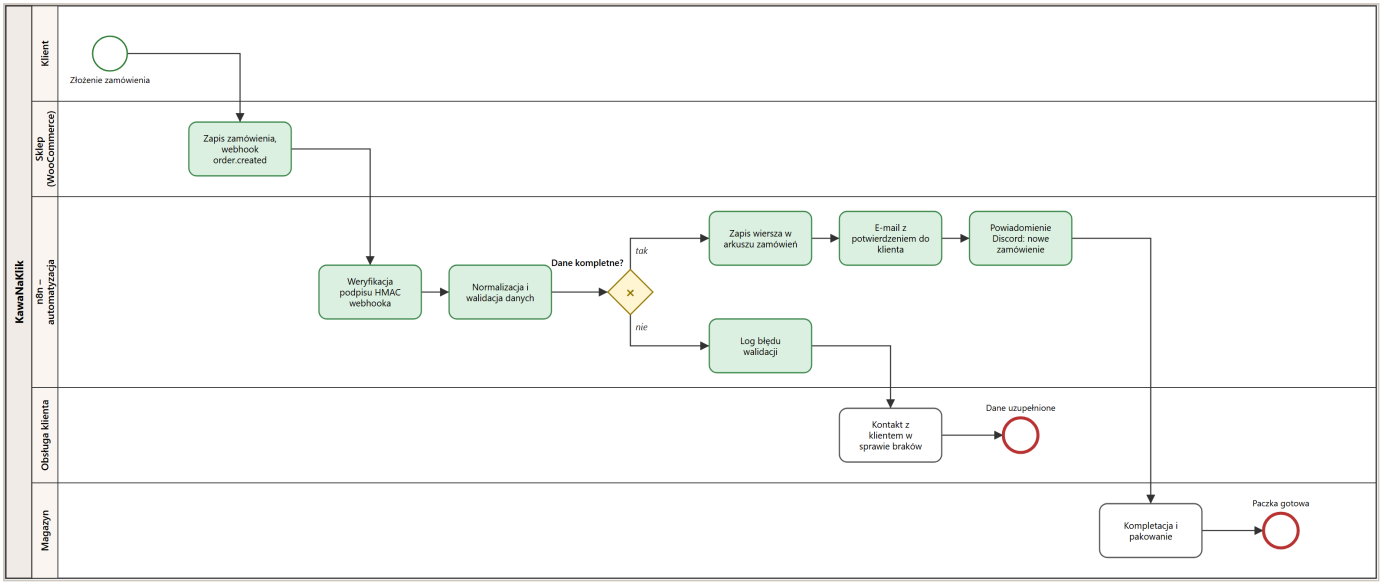
Logi mają dwie warstwy. **Historia wykonań n8n** zawiera pełne dane wejściowe i wyjściowe każdego węzła, czyli materiał do diagnozy. **Pliki CSV** to krótki, czytelny dla człowieka dziennik zdarzeń (czas, źródło, opis, szczegóły), który można filtrować w Excelu.



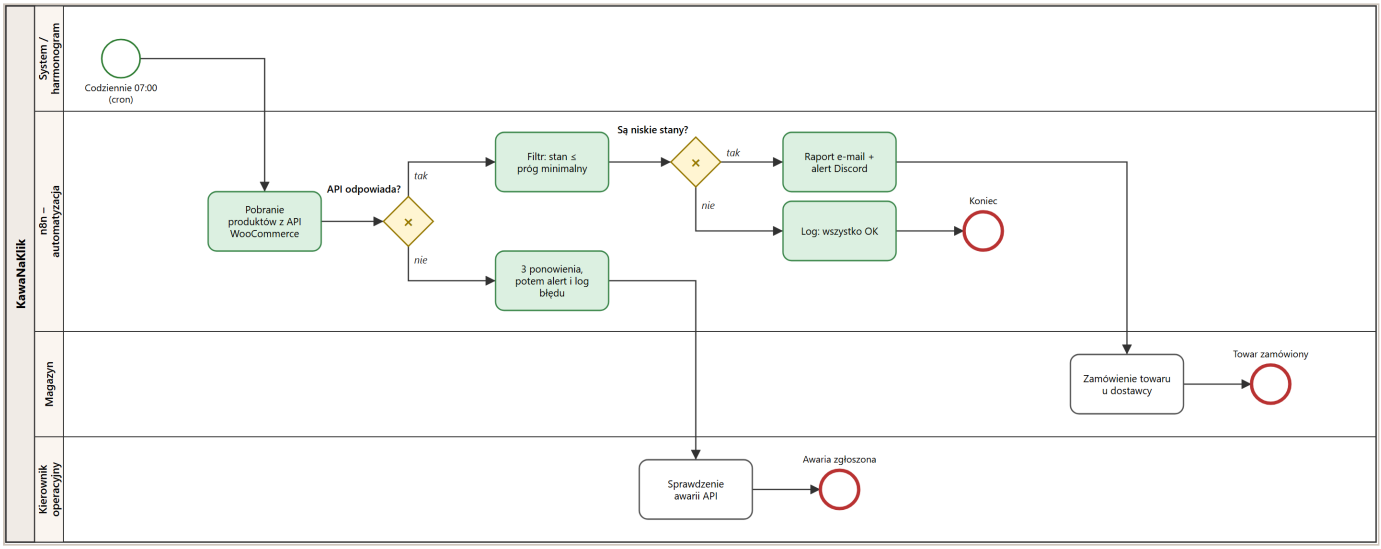

Rys. A5. Historia wykonań po uruchomieniu zestawu testów – jedno wykonanie z błędem (T5), obsłużone przez WF3.

# A7. Diagramy procesu docelowego (TO-BE)

krok automatyczny (n8n / system) | krok człowieka




Rys. A6. TO-BE 1 – realizacja zamówienia. Względem AS-IS (LAB02, rozdz. A3.1) znika ręczne przepisywanie zamówienia do arkusza i ręczne potwierdzenia. Człowiek zajmuje się tylko wyjątkami (braki w danych) oraz pakowaniem.




Rys. A7. TO-BE 2 – kontrola stanów magazynowych. Codzienne ręczne przeglądanie stanów zastępuje raport o 07:00. Awaria API kończy się alertem zamiast cichym brakiem raportu.

# A8. Testy

Wszystkie przypadki uruchamia jeden skrypt `kod/testy/testy_n8n.sh`. Zamówienia tworzy przez REST API WooCommerce, więc sklep sam wysyła prawdziwy webhook. Przypadki T2b i T3 wysyłają żądanie bezpośrednio na adres webhooka, a T7 zatrzymuje stronę sklepu przez API aplikacji Local. Pełny wynik: `testy/wyniki_testow.txt`, pliki CSV po testach: `testy/*.csv`.

| # | Przypadek | Oczekiwany wynik | Wynik | |
|---|---|---|---|---|
| T1 | Poprawne zamówienie nr 156 (2 pozycje, przelew) | Wiersz w arkuszu, e-mail do klienta, wiadomość Discord, HTTP 200 | Wiersz w `arkusz_zamowien.csv`, e-mail „Potwierdzenie zamówienia nr 156", wiadomość „🛒 Nowe zamówienie nr 156…", wykonanie w 115 ms (rys. A8, A14) | ✓ |
| T2 | Zamówienie nr 157 bez e-maila i adresu | Brak wpisu w arkuszu, log, alert do obsługi, HTTP 422 | Log: „brak adresu e-mail, niepełny adres", e-mail „⚠ Zamówienie 157 wymaga uzupełnienia danych", alert Discord (rys. A9, A15) | ✓ |
| T2b | Podpisany webhook z adresem `jan.kowalski@poczta` | Wykrycie błędnego formatu e-maila, HTTP 422 | `422 {"ok":false,"problemy":["niepoprawny adres e-mail"]}`, wpis w logu | ✓ |
| T3 | Fałszywy podpis HMAC | Odrzucenie, log, HTTP 401 | `401 {"ok":false,"blad":"nieprawidłowy podpis webhooka"}`, log „nieprawidłowy podpis" (rys. A10) | ✓ |
| T3b | Żądanie bez podpisu | Odrzucenie, log, HTTP 401 | 401, log „brak podpisu" | ✓ |
| T4 | Ping WooCommerce (`webhook_id=1`) | HTTP 200 bez przetwarzania zamówienia | `200 {"ok":true,"ping":true}` | ✓ |
| T5 | Zamówienie nr 158 z notatką `#TEST-BLAD` (symulowany wyjątek) | Wykonanie WF1 z błędem, uruchomienie WF3 | WF1 #55 – błąd w węźle „Weryfikacja i normalizacja". WF3 #56 zapisał log z numerem wykonania, wysłał alert Discord i e-mail do administratora (rys. A11, A12) | ✓ |
| T6 | Kontrola stanów (GET na żądanie = to samo co cron) | Raport e-mail i alert z produktami o stanie ≤ 5 | 4 z 11 produktów: KNK-POR 0, KNK-ESP-1000 0, KNK-MEX-250M 1, KNK-MLY 5. E-mail „📦 Niskie stany magazynowe: 4 produkty" (rys. A16, A17) | ✓ |
| T7 | Awaria API: strona sklepu zatrzymana w Local | 3 próby, potem log + alert, brak raportu | Węzeł API pracował 12,4 s (3 próby, odpowiedź *Bad gateway*). Wyjście „Error" → log, alert Discord „🔴 Kontrola stanów nie powiodła się…", e-mail do kierownika (rys. A13) | ✓ |

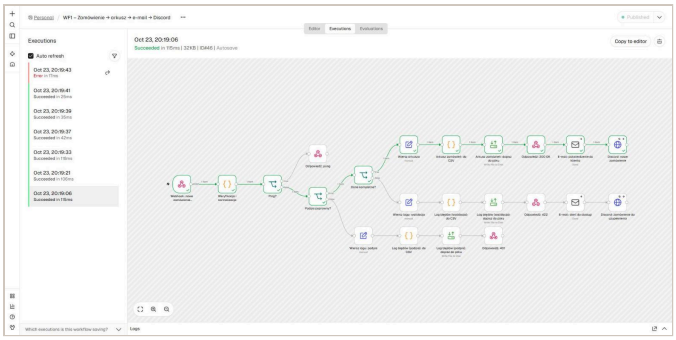
## Log błędów po testach (`log_bledow.csv`)

```
czas;zrodlo;opis;szczegoly
2026-10-23 20:18:22;WF2 – API WooCommerce;API WooCommerce nie odpowiada (3 próby, co 3 s);Bad gateway - the service failed to handle your request
2026-10-23 20:19:21;WF1 – walidacja zamówienia;Zamówienie 157: brak adresu e-mail, niepełny adres;Ewa Niepełna <>
2026-10-23 20:19:33;WF1 – walidacja zamówienia;Zamówienie 9001: niepoprawny adres e-mail;Jan Kowalski <jan.kowalski@poczta>
2026-10-23 20:19:37;WF1 – webhook zamówień;Odrzucono webhook: nieprawidłowy podpis;topic: order.created, zamówienie: 999
2026-10-23 20:19:39;WF1 – webhook zamówień;Odrzucono webhook: brak podpisu;topic: -, zamówienie: 998
2026-10-23 20:19:43;WF1 – Zamówienie → arkusz → e-mail → Discord;Symulowany błąd przetwarzania zamówienia 158 (test WF3) [line 27];węzeł: Weryfikacja i normalizacja, wykonanie #55
```

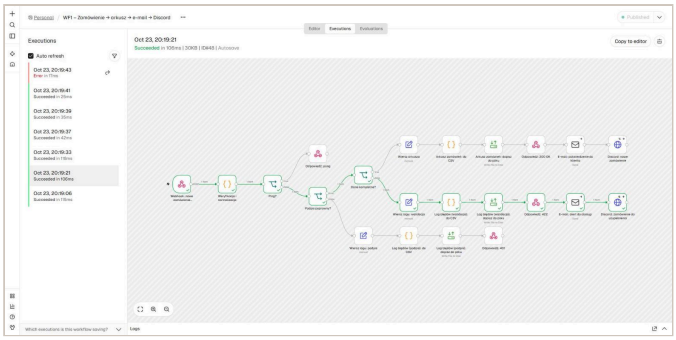
## Arkusz zamówień i log kontroli stanów

```
zapisano;numer;data_zamowienia;klient;email;telefon;adres;pozycje;razem;waluta;platnosc;status
2026-10-23 20:19:06;156;2026-10-23T20:19:05;Anna Testowa;anna.testowa@example.com;+48 500 100 200;ul. Kawowa 12, 30-001, Kraków;"2× Etiopia Yirgacheffe 250 g; 1× Filtry papierowe 100 szt.";119.70;PLN;Przelew bankowy;processing

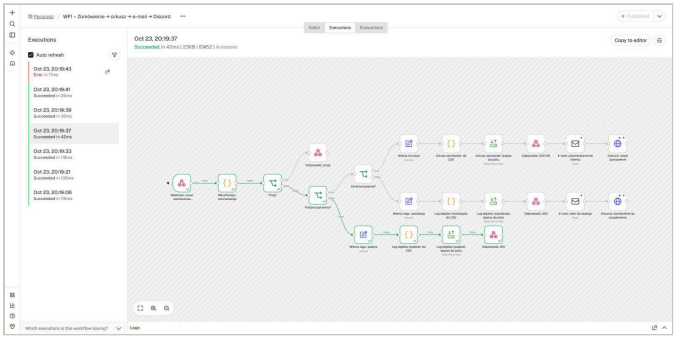
czas;sprawdzono;niskie;produkty
2026-10-23 20:19:54;11;4;KNK-POR=0, KNK-ESP-1000=0, KNK-MEX-250M=1, KNK-MLY=5
```



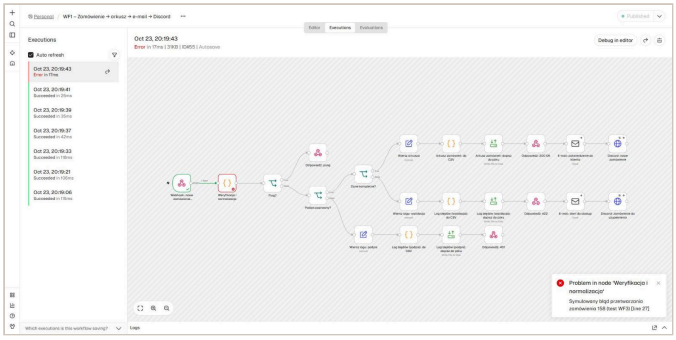
Rys. A8. T1 – wykonanie ścieżki poprawnej (zielone węzły).



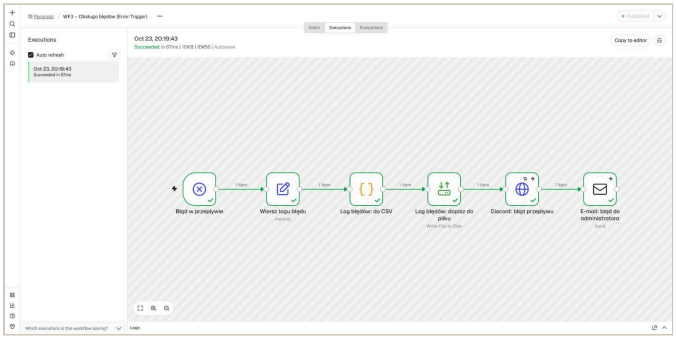
Rys. A9. T2 – ścieżka „dane niekompletne”.



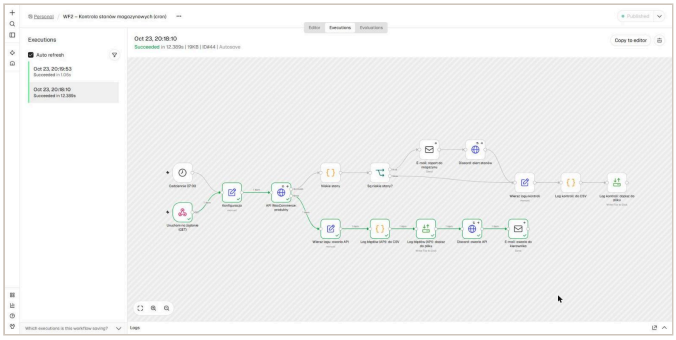
Rys. A10. T3 – zły podpis: log i odpowiedź 401.



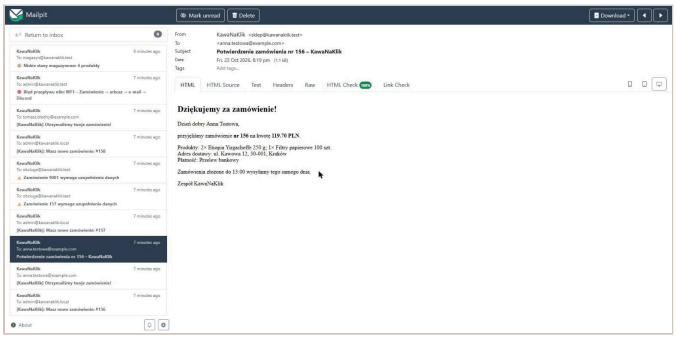
Rys. A11. T5 – wyjątek w węźle Code; obok wykonanie WF3 (rys. A12).



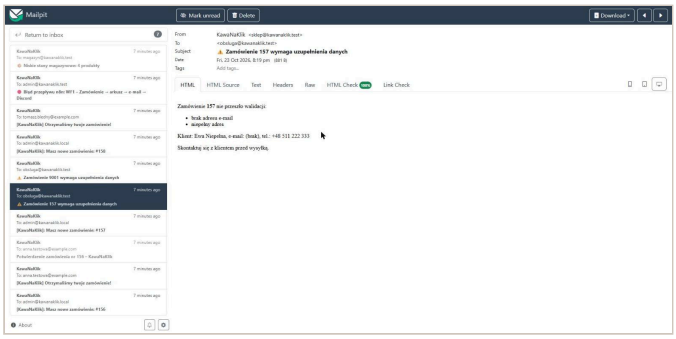
Rys. A12. T5 – WF3 przechwycił błąd: log, Discord, e-mail.



Rys. A13. T7 – API nie odpowiada: wyjście „Error” węzła HTTP po 3 próbach.



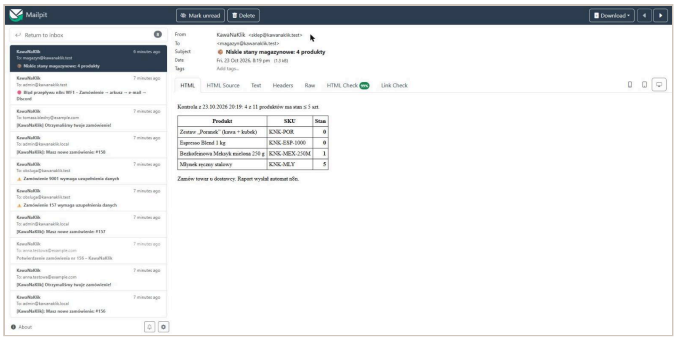
Rys. A14. T1 – e-mail z potwierdzeniem do klienta (Mailpit).



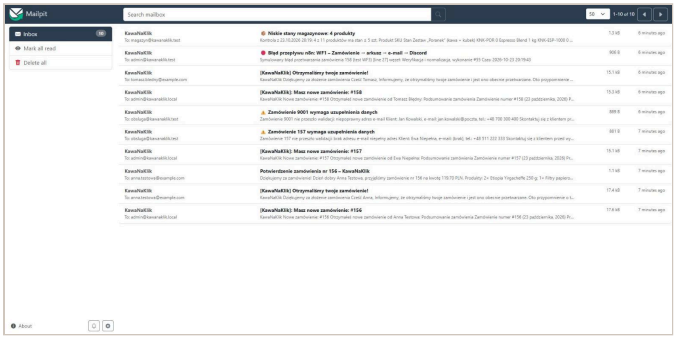
Rys. A15. T2 – alert do obsługi klienta.



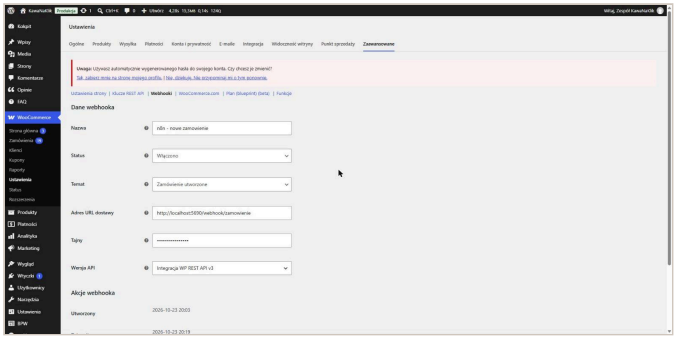
Rys. A16. T6 – wykonanie WF2 (ścieżka „są niskie stany").



Rys. A17. T6 – raport niskich stanów do magazynu.



Rys. A18. Skrzynka Mailpit po testach: e-maile z n8n obok standardowych e-maili WooCommerce.



Rys. A19. Webhook w WooCommerce (sekret zamaskowany na zrzucie).

## A9. Napotkane problemy

| # | Problem | Przyczyna i rozwiązanie |
|---|---|---|
| 1 | Instalacja `npm install n8n` przerwana przez brak pamięci RAM | n8n ma ok. 1500 zależności, a instalacja npm zajmuje kilka GB pamięci. Użyliśmy instalacji n8n wykonanej wcześniej przez **pnpm**, która zużywa mniej pamięci. Nasza instancja ma osobny katalog danych i osobne porty (5690, broker zadań 5691), więc nie koliduje z inną instancją na porcie 5678. |
| 2 | WooCommerce nie wysyłał webhooka: „Podano nieprawidłowy adres URL” | Ochrona przed SSRF w `wp_http_validate_url()`: domyślnie blokowane są adresy lokalne i porty inne niż 80/443/8080. Wtyczka MU dopuszcza tylko `localhost:5690` (filtry `http_request_host_is_external` i `http_allowed_safe_ports`). Zmiana działa wyłącznie na środowisku deweloperskim. |
| 3 | Zmiany w PHP widoczne z opóźnieniem | Na stronie działa OPcache sprawdzający pliki co 60 s. Po edycji wtyczki trzeba odczekać minutę albo zrestartować stronę. |
| 4 | Dopisywane wiersze CSV sklejały się w jedną linię, a każdy zaczynał się znakiem BOM | Węzeł *Convert to File* generuje kompletny plik (BOM, brak końca linii), a nie pojedynczy wiersz. Zastąpiliśmy go węzłem Code, który składa wiersz CSV z poprawnym cytowaniem pól (`;`, cudzysłowy, nowe linie) i końcem linii. |
| 5 | Pierwsze wykonanie po restarcie Mailpit „wisiało” ok. 2 min, a sklep czekał na odpowiedź webhooka | Nazwa `localhost` najpierw trafiała na IPv6, gdzie połączenie SMTP czekało do domyślnego limitu nodemailera (120 s). Rozwiązania: SMTP na `127.0.0.1`, odpowiedź do WooCommerce przed wysyłką e-maila, limit czasu wykonania przepływu 120 s. |
| 6 | Restart strony w teście T7 kasował e-maile z wcześniejszych testów | Mailpit w Local trzyma wiadomości w pamięci i restartuje się razem ze stroną. T7 przenieśliśmy na początek zestawu testów. E-mail o awarii API potwierdza historia wykonań (węzeł „E-mail: awaria do kierownika” zakończony sukcesem). |
| 7 | Brak kont Google i Discord w środowisku laboratoryjnym | Arkusz jako plik CSV i zastępczy webhook Discorda (WF0) z identycznym formatem wiadomości. Przejście na usługi docelowe to podmiana węzła i ustawienie poświadczenia. |
| 8 | WooCommerce wyłącza webhook po 5 kolejnych nieudanych dostawach | Odpowiedzi 401 i 422 liczą się jako nieudane dostawy. Licznik zeruje się po pierwszej udanej dostawie. W produkcji zamówienie z brakami warto przyjmować z kodem 200 i oznaczać jako „do uzupełnienia”, żeby seria błędnych zamówień nie wyłączyła integracji. |

## A10. Efekt i wnioski

Według wskaźników bazowych z LAB02 (rozdz. A6) ręczne przepisanie zamówienia do arkusza trwa 2,5 min (1,96 zł). Przy 104 zamówieniach dziennie WF1 oszczędza ok. **4,3 h pracy dziennie** i ok. **6 100 zł miesięcznie**. Do tego dochodzą natychmiastowe potwierdzenie dla klienta i powiadomienie zespołu. WF2 zastępuje codzienne ręczne przeglądanie stanów i, co ważniejsze, nie zapomina o nim. Przy awarii sklepu informuje o niej, zamiast po cichu nie wysłać raportu.

- Najwięcej pracy wymagały ścieżki błędne, a nie ścieżka poprawna. Z 44 węzłów w WF1–WF3 mniej więcej połowa obsługuje wyjątki, logowanie i alerty.
- Automat bez AI dobrze radzi sobie z danymi o stałej strukturze: zamówienie z WooCommerce ma zawsze te same pola, więc reguły walidacji są proste i przewidywalne.
- Bezpieczeństwo integracji wymaga świadomych decyzji:
  - podpis HMAC na webhooku,

    - sekrety poza repozytorium (zmienne środowiskowe, poświadczenia n8n),
    - ograniczenie zapisu plików do jednego katalogu,
    - dopuszczenie wyjątku SSRF tylko dla jednego adresu.
- Te przepływy są fundamentem dla kolejnego etapu. Webhook, walidacja, logowanie i obsługa błędów zostają, a w miejscach z danymi nieustrukturyzowanymi pojawi się model językowy (LLM). Przykłady: klasyfikacja i szkic odpowiedzi na e-maile klientów, odczyt faktur PDF.

# Część B – CMS

Laboratorium 2: ACF, własne typy treści i bloki

## B1. Realizacja zadań

| Zadanie | Realizacja | Status |
|---|---|---|
| Katalog `acf-json` w motywie, pola w repozytorium, synchronizacja u drugiej osoby | 6 grup pól zapisanych jako pliki JSON w `acf-json/`. Na stronie bez tych grup w bazie panel pokazał „Synchronizacja jest dostępna (6)". Po synchronizacji kolumna „Lokalny JSON" ma status „Zapisana" (rys. B1–B3). | ✓ |
| Min. 4 własne bloki ACF, strona główna złożona tylko z nich | 5 bloków: **Hero**, **Przewagi (USP)**, **Opinie klientów** (z CPT), **FAQ** (z CPT + JSON-LD), **Z bloga**. `front-page.php` wywołuje już tylko `the_content()`, a strona główna składa się wyłącznie z tych 5 bloków. | ✓ |
| Min. 1 CPT z polami ACF, archiwum i widokiem pojedynczym | CPT `opinia` z polami ocena, miasto i produkt, archiwum `/opinie/` (`archive-opinia.php`) i widokiem pojedynczym (`single-opinia.php`). Dodatkowo CPT `faq` dla bloku FAQ. | ✓ |
| Lista dostępnych bloków ograniczona filtrem | `allowed_block_types_all`: 5 bloków motywu + akapit, nagłówek, lista, obrazek. Strony WooCommerce (koszyk, zamówienie, konto) zachowują swoje bloki (rys. B12). | ✓ |
| Dane globalne edytowalne z panelu, wyświetlane w stopce | Customizer → „Dane sklepu": pasek promocji, telefon, e-mail, adres, Facebook, Instagram. Wyświetlane w stopce i w pasku nad nagłówkiem (rys. B7, B13). | ✓ |
| Każda wyświetlana wartość escapowana, brak `the_field()` | W każdym szablonie jest `get_field()` + `esc_html` / `esc_attr` / `esc_url` / `wp_kses_post` / `wp_get_attachment_image`. Polecenie `grep -rn "the_field(" themes/kawanaklik` nie zwraca żadnego wyniku. | ✓ |
| Commit i tag `lab2` | Commit w repozytorium zespołu, tag `lab2`. | ✓ |

> **ACF → Secure Custom Fields.** Bloki ACF są funkcją **ACF PRO**. W darmowym ACF 6.8 bloki rejestrowały się bez funkcji renderującej, więc strona główna była pusta. Zamiast płatnej licencji użyliśmy **Secure Custom Fields (SCF)**, czyli darmowego forka ACF utrzymywanego przez WordPress.org. SCF ma identyczne API (`get_field()`, `acf-json`, `block.json` z kluczem `acf`) i obsługuje bloki. Kod motywu nie wymagał żadnych zmian. Repeatera i strony opcji celowo nie używamy, żeby motyw działał także z darmowym ACF: FAQ jest osobnym CPT, a dane globalne są w Customizerze.

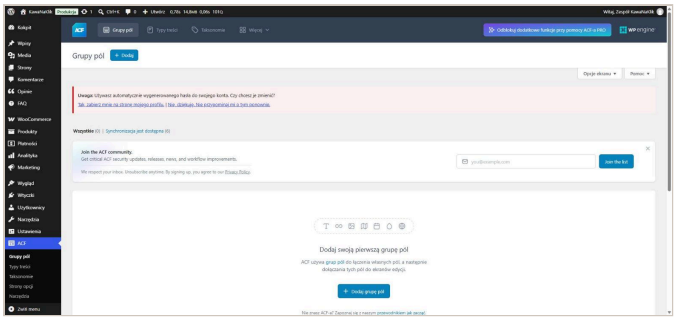
## B2. ACF Local JSON – pola w repozytorium

W motywie jest katalog `acf-json/`. Każdy zapis grupy pól w panelu zapisuje też plik `group_*.json`, który commitujemy razem z kodem. Pliki mają czytelne klucze (`group_knk_blok_hero`, `field_knk_hero_title`), więc różnice w `git diff` łatwo zrozumieć.

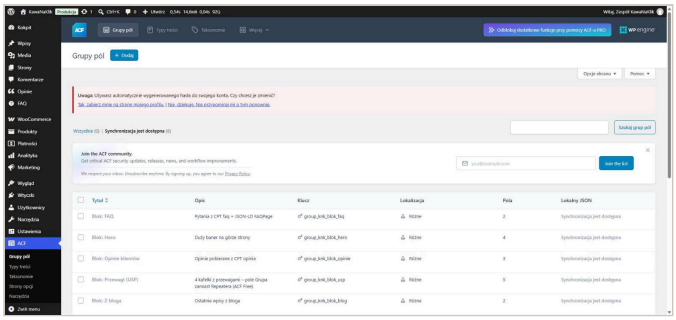
**Test synchronizacji („druga osoba z zespołu"):**

1. Na stronie wgraliśmy motyw z katalogiem `acf-json`. Baza danych nie miała żadnej grupy pól, czyli sytuacja jak u kolegi po `git pull`.

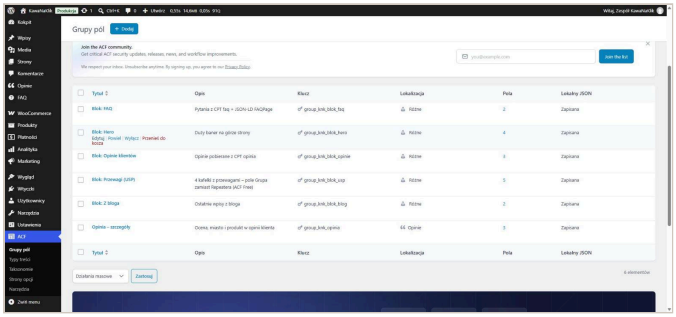
2. W ACF → Grupy pól pojawiło się „Synchronizacja jest dostępna (6)”. Pola działają od razu, bo ACF czyta je z JSON-a, ale nie da się ich edytować w panelu, dopóki nie trafią do bazy.
3. Po synchronizacji grupy są w bazie, a kolumna „Lokalny JSON” pokazuje „Zapisana”, czyli baza i plik są zgodne.



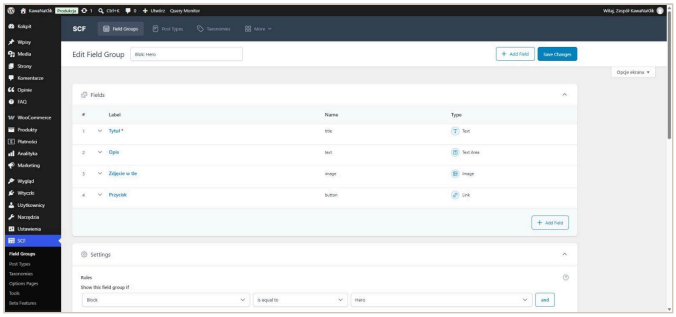
Rys. B1. Świeża baza: „Synchronizacja jest dostępna (6)”.



Rys. B2. Lista grup z plików acf-json czekających na synchronizację.



Rys. B3. Po synchronizacji: 6 grup, Lokalny JSON = „Zapisana”.



Rys. B4. Grupa „Blok: Hero”: 4 pola, reguła lokalizacji Blok = Hero.

## B3. Własne typy treści

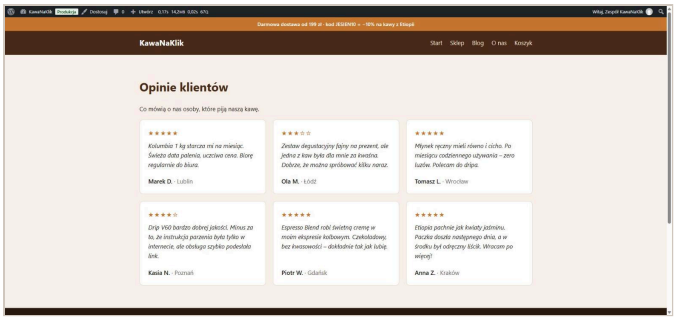
Oba typy treści rejestrujemy w kodzie motywu ( `inc/post-types.php` , akcja `init` ), a nie w panelu, żeby rejestracja była w repozytorium. Po zmianach wykonujemy `wp rewrite flush` , bo bez tego adres `/opinie/` zwraca 404.

```
register_post_type( 'opinia', array(
    'labels'       => array( 'name' => __( 'Opinie', 'kawanaklik' ), ... ),
    'public'       => true,
    'has_archive'  => true,                    // archiwum pod /opinie/
    'rewrite'      => array( 'slug' => 'opinie' ),
    'menu_icon'    => 'dashicons-format-quote',
    'supports'     => array( 'title', 'editor', 'thumbnail' ),
    'show_in_rest' => true,                    // edytor blokowy + REST API
) );
// faq: public => false, show_ui => true – pytania widać tylko w bloku FAQ, nie mają własnych adresów.
```

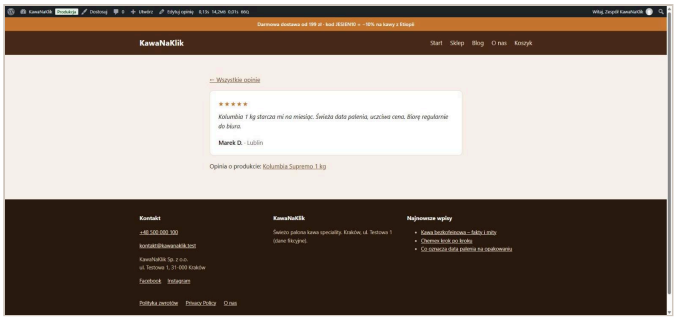
Grupa „Opinia – szczegóły” ma trzy pola:

- `ocena` – lista wyboru 1–5,
- `miasto` – tekst,
- `produkt` – Post Object na produkt WooCommerce, format zwracany: ID.

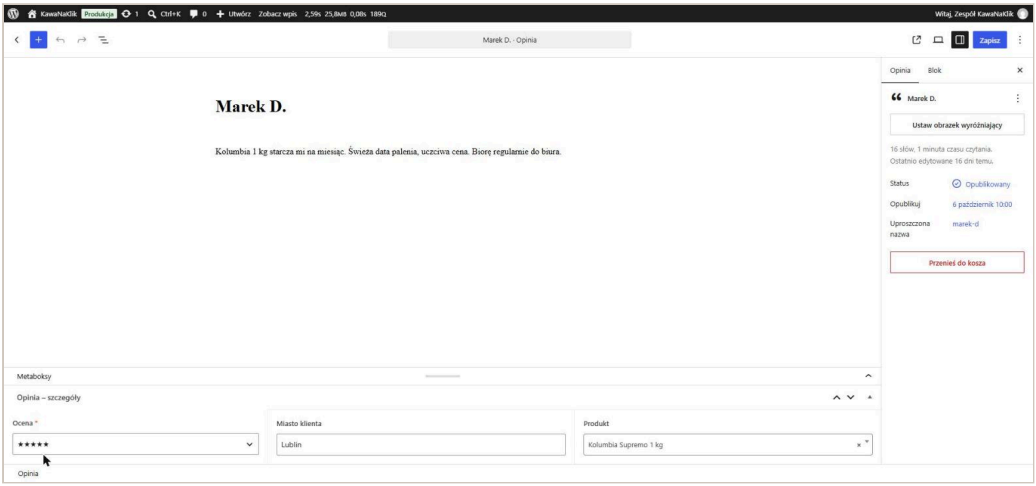
Kartę opinii ( `template-parts/opinia-card.php` ) używają trzy miejsca: blok, archiwum i widok pojedynczy.




Rys. B5. Archiwum /opinie/ (archive-opinia.php).




Rys. B6. Pojedyncza opinia (single-opinia.php) z linkiem do produktu.




Rys. B7. Edycja opinii – pola ACF: ocena, miasto, produkt.

## B4. Bloki ACF

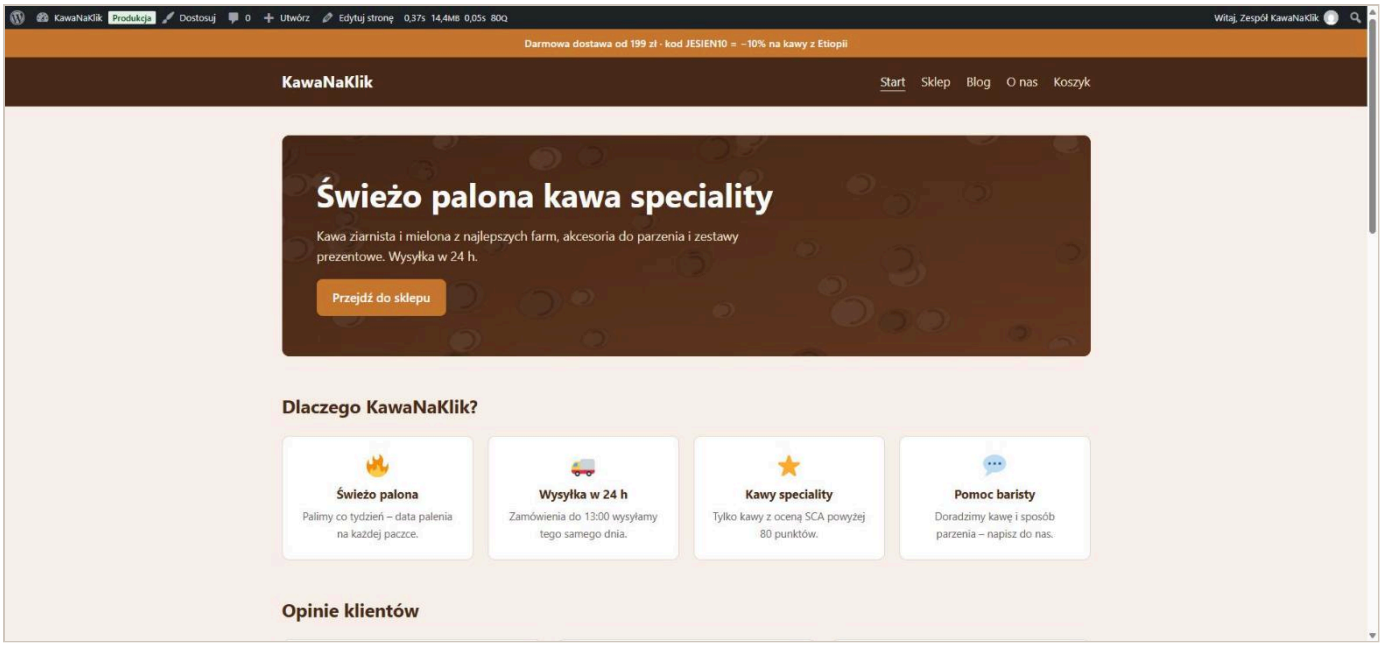
Każdy blok to katalog w `blocks/` z plikami `block.json`, `render.php` i `style.css`. `inc/blocks.php` rejestruje automatycznie każdy katalog z plikiem `block.json`. Bloki mają własną kategorię „KawaNaKlik" w okienku wstawiania. Style bloków ładują się tylko na stronach, na których blok występuje.

| Blok | Pola ACF | Źródło danych |
|---|---|---|
| `acf/hero` | title, text, image (ID), button (link) | pola bloku |
| `acf/usp` | title + 4 × grupa (icon, title, text) | pola bloku (pole Grupa zamiast Repeatera) |
| `acf/opinie` | title, count, min_rating | WP_Query po CPT `opinia` z `meta_query` ocena ≥ min |
| `acf/faq` | title, count | CPT `faq` + dane strukturalne **FAQPage** (JSON-LD) |
| `acf/blog` | title, count | WP_Query po wpisach + `wp_reset_postdata()` |

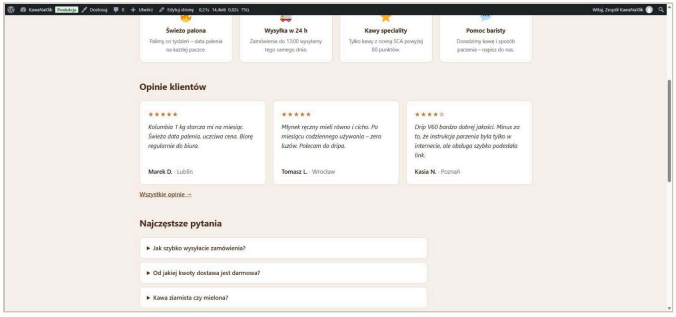
```
// blocks/hero/block.json
{
  "apiVersion": 3,
  "name": "acf/hero",
  "title": "Hero",
  "category": "kawanaklik",
  "icon": "cover-image",
  "acf": {
    "mode": "preview",
    "renderTemplate": "render.php"
  },
  "supports": { "anchor": true, "align": false },
  "style": "file:./style.css"
}
```

```
// blocks/hero/render.php (fragment)
$title  = get_field( 'title' );
$image  = get_field( 'image' );   // ID załącznika
$button = get_field( 'button' );  // url, title,
target
...
<?php echo wp_get_attachment_image( $image,
'full',
      false, array( 'class' => 'hero__bg' ) ); ?>
<h1 class="hero__title"><?php echo esc_html(
$title ); ?></h1>
<a class="button"
   href="<?php echo esc_url( $button['url'] ); ?>"
   target="<?php echo esc_attr( $button['target']
?: '_self' ); ?>">
  <?php echo esc_html( $button['title'] ); ?></a>
```

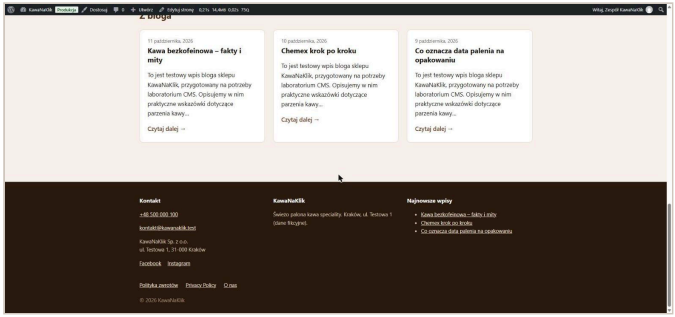
Blok FAQ dodaje do strony `<script type="application/ld+json">` z typem `FAQPage`. Kodujemy go przez `wp_json_encode()` z flagą `JSON_HEX_TAG`: znaki < i > z treści pytań zamieniają się na `<` / `>`, więc treść nie może zamknąć znacznika `<script>`.



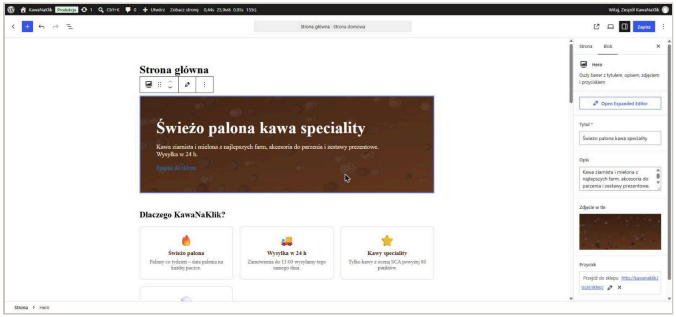

Rys. B8. Strona główna: pasek promocji (Customizer), blok Hero (zdjęcie z pola image) i blok Przewagi.



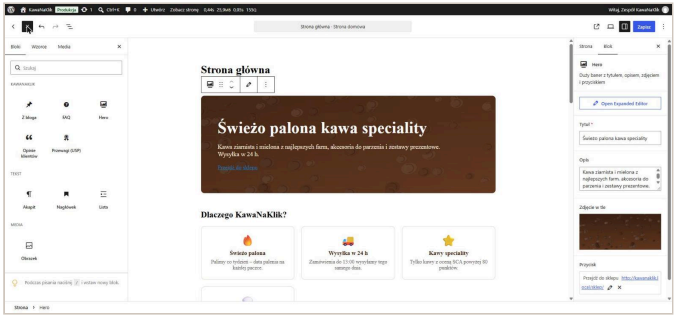

Rys. B9. Bloki Opinie (z CPT, ocena ≥ 4) i FAQ (z CPT).




Rys. B10. Blok Z bloga i stopka z danymi globalnymi.




Rys. B11. Edytor: blok Hero i jego formularz pól w panelu bocznym.




Rys. B12. Okienko wstawiania ograniczone do 5 bloków motywu i 4 podstawowych.

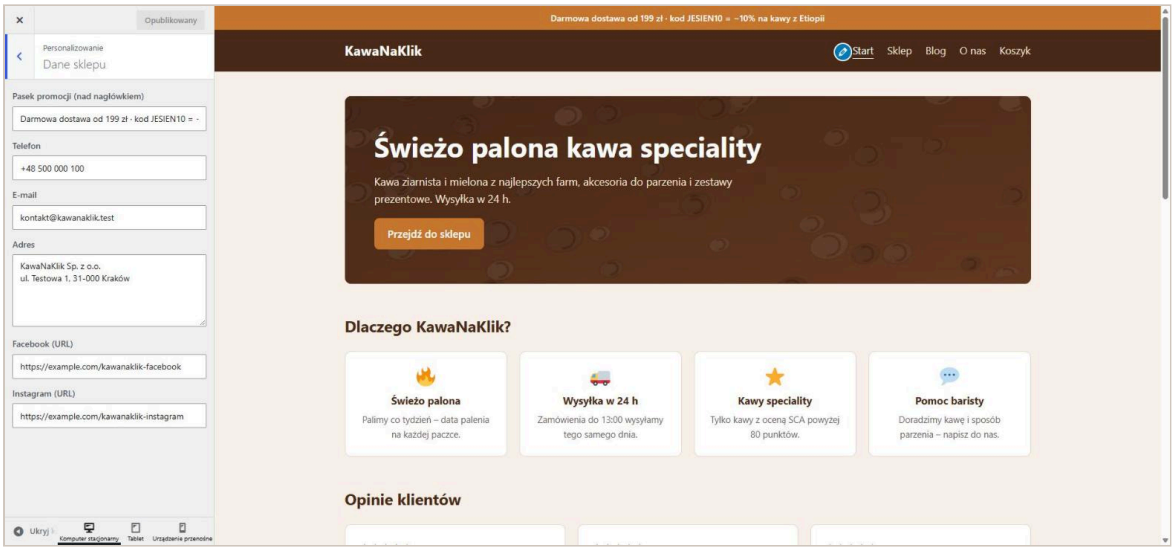
## B5. Ograniczenie listy bloków i dane globalne

```
// inc/blocks.php
function kawanaklik_allowed_blocks( $allowed,
$context ) {
    // Strony WooCommerce zachowują swoje bloki
    if ( ! empty( $context->post ) && in_array(
        $context->post->ID, $woo_pages, true ) ) {
        return $allowed;
    }
    return array( 'acf/hero', 'acf/usp',
'acf/opinie',
        'acf/faq', 'acf/blog', 'core/paragraph',
        'core/heading', 'core/list', 'core/list-
item',
        'core/image' );
}
add_filter( 'allowed_block_types_all',
    'kawanaklik_allowed_blocks', 10, 2 );
```

```
// inc/customizer.php – ACF Free nie ma strony
opcji
$wp_customize->add_setting( 'kawanaklik_telefon',
array(
    'sanitize_callback' => 'sanitize_text_field',
) );
// facebook/instagram: esc_url_raw, e-mail:
sanitize_email

// footer.php – escapowanie przy wyświetlaniu
<a href="<?php echo esc_url( 'tel:' . $tel_cyfry
); ?>">
  <?php echo esc_html( $telefon ); ?></a>
<p><?php echo nl2br( esc_html( $adres ) ); ?></p>
```

Dane są chronione dwa razy: **sanitize** przy zapisie w Customizerze i **escape** przy wyświetlaniu.




Rys. B13. Customizer → Dane sklepu: pasek promocji, kontakt, media społecznościowe.

# B6. Odpowiedzi na pytania

## B6.1. Schemat grup pól

Schemat wygenerowaliśmy automatycznie z plików `acf-json/*.json`, więc zawsze odpowiada temu, co jest w repozytorium. Obrazki mają format zwracany **ID**, więc `wp_get_attachment_image()` sam dodaje `srcset`, `alt` i lazy loading. Link zwraca tablicę (url, title, target). Post Object zwraca ID produktu.

**Grupy pól ACF – motyw KawaNaKlik (źródło: acf-json/)**

**Blok: Hero** — Blok = acf/hero

| nazwa pola | typ | format / wartość w PHP |
|---|---|---|
| title | text | tekst |
| text | textarea | tekst |
| image | image | zwraca: id |
| button | link | zwraca: array |

**Blok: Opinie klientów** — Blok = acf/opinie

| nazwa pola | typ | format / wartość w PHP |
|---|---|---|
| title | text | tekst |
| count | number | liczba 1–12 |
| min_rating | select | zwraca: value |

**Blok: FAQ** — Blok = acf/faq

| nazwa pola | typ | format / wartość w PHP |
|---|---|---|
| title | text | tekst |
| count | number | liczba 1–20 |

**Opinia – szczegóły** — Typ treści = opinia

| nazwa pola | typ | format / wartość w PHP |
|---|---|---|
| ocena | select | zwraca: value |
| miasto | text | tekst |
| produkt | post_object | zwraca: id (product) |

**Blok: Przewagi (USP)** — Blok = acf/usp

| nazwa pola | typ | format / wartość w PHP |
|---|---|---|
| title | text | tekst |
| usp_1 | group | tablica pól |
| └ icon | select | zwraca: value |
| └ title | text | tekst |
| └ text | textarea | tekst |
| usp_2 | group | tablica pól |
| └ icon | select | zwraca: value |
| └ title | text | tekst |
| └ text | textarea | tekst |
| usp_3 | group | tablica pól |
| └ icon | select | zwraca: value |
| └ title | text | tekst |
| └ text | textarea | tekst |
| usp_4 | group | tablica pól |
| └ icon | select | zwraca: value |
| └ title | text | tekst |
| └ text | textarea | tekst |

**Blok: Z bloga** — Blok = acf/blog

| nazwa pola | typ | format / wartość w PHP |
|---|---|---|
| title | text | tekst |
| count | number | liczba 1–9 |

Rys. B14. Grupy pól: nazwa pola, typ, format zwracany, reguła lokalizacji.

## B6.2. ACF a page builder

| | ACF + bloki motywu | Page builder (Elementor, Divi…) |
|---|---|---|
| **Klient** | Prosty formularz: tytuł, opis, zdjęcie, przycisk. Nie da się „zepsuć" wyglądu. | Pełna swoboda kolorów, odstępów i czcionek, ale łatwo o niespójny wygląd po kilku miesiącach. |
| | Spójny wygląd na każdej stronie – identyczne bloki, te same style. | Każda strona może wyglądać inaczej. Trudno utrzymać jedną identyfikację wizualną. |
| | Szybka strona (lekki HTML), co oznacza lepsze SEO i konwersję. | Ciężki kod (wiele zagnieżdżonych , duże CSS/JS), wolniejsze ładowanie. |

| | ACF + bloki motywu | Page builder (Elementor, Divi…) |
|---|---|---|
| **Programista** | Treść jest oddzielona od prezentacji. Można przebudować wygląd (PHP + CSS) bez ruszania treści. | Treść jest zapisana razem z układem w formacie buildera. Zmiana motywu lub buildera oznacza przepisanie stron. |
| | Wszystko w repozytorium: kod, pola (Local JSON), rejestracja CPT. Review i wersjonowanie przez Git. | Układ siedzi w bazie danych. Trudno go wersjonować i przenosić między środowiskami. |
| | Pełna kontrola nad HTML, dostępnością, escapowaniem i wydajnością. | Zależność od wtyczki zewnętrznej (aktualizacje, licencje, luki bezpieczeństwa). Ograniczona kontrola nad kodem. |

### B6.3. Test XSS: `<script>alert(1)</script>` w tytule bloku Hero

Wpisaliśmy ten tekst w pole „Tytuł" bloku Hero i zapisaliśmy stronę. **Nic się nie wykonało.** Tytuł wyświetla się dosłownie jako tekst, zarówno w podglądzie edytora, jak i na stronie (rys. B15–B16). Dzieje się tak, bo `render.php` wypisuje tytuł przez `esc_html()`, które zamienia znaki specjalne HTML na encje. Przeglądarka dostaje więc tekst, a nie znacznik:

```
z esc_html():   <h1 class="hero__title">&lt;script&gt;alert(1)&lt;/script&gt;</h1>
bez esc_html(): <h1 class="hero__title"><script>alert(1)</script></h1>
```

**Bez `esc_html()`** (sprawdziliśmy na kopii motywu, pobierając stronę przez `curl`, a nie w przeglądarce) do HTML trafia działający znacznik `<script>`:

- Przeglądarka każdego odwiedzającego wykonałaby skrypt, czyli doszłoby do ataku **stored XSS**.
- Zamiast `alert(1)` atakujący mógłby wykraść ciasteczka sesji administratora, podmienić przycisk „Kup" na fałszywą bramkę płatności albo wykonywać akcje w panelu w imieniu zalogowanego admina.

Pole ACF ani baza danych przed tym nie chronią. Wartość jest zapisana bezpiecznie w JSON-ie bloku (